

Tab 1

NightWise App Build Roadmap -

Simply put-

It'll be a no-login app that compares route options at night, scores each route using activity signals around it, explains the tradeoff in plain English, and then opens the route the user chooses in Google Maps.

What We Are Not Building First

We are not copying Safetipin's city-safety audit feature, for crowdsourcing data.

We are not claiming any route is guaranteed safe.

We are not building full turn-by-turn navigation inside our app.

We are not building a community reporting network in version one.

The First Demo Should Prove

A user can enter a destination.

The app can get 2 to 4 route options.

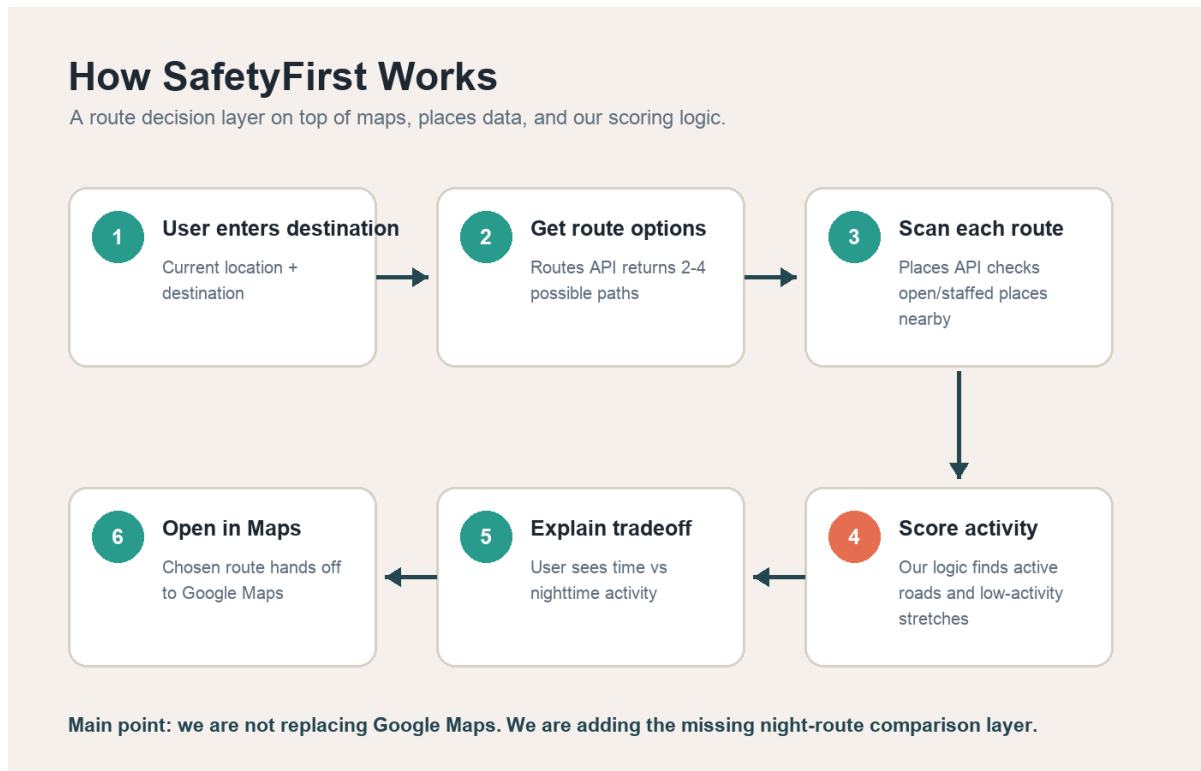
The app can scan what exists around each route.

The app can identify which route has more nighttime activity.

The app can explain the recommendation in one or two lines.

The user can open their chosen route in Google Maps.

Simple Product Flow



What All We Need

1. Google Cloud access

This is needed to use Google Maps APIs. The engineer needs a Google Cloud project with billing enabled.

2. Google Maps API key

This is the master key the app uses to call Google services. It must be restricted so it can only be used by our app and only for the APIs we enable.

3. Maps JavaScript API

This shows the actual map inside our web app.

4. Routes API

This gets the available route options between origin and destination. It gives time, distance, route shape, and traffic-aware alternatives.

5. Places API

This checks what exists near each route: shops, cafes, restaurants, petrol pumps, hospitals, hotels, police stations, pharmacies, public transport points, and other staffed places.

Also check if this is the main road or a shortcut going through internal roads.

6. Places Autocomplete API

This helps the user type their destination without entering exact addresses manually.

7. Geocoding API

This converts addresses into latitude and longitude. We may need it when a user types an address instead of selecting a place.

8. Browser location permission

This lets the app use the user's current location. The user must explicitly allow it.

9. Backend server

This protects private API keys and runs the route analysis. The frontend should not directly call every private API.

11. Database later

Supabase or Firebase is only needed when we add live tracking, temporary tracking links, community reports, or route feedback.

Important Data Reality Check

Open shops, cafes, restaurants, hotels, hospitals, petrol pumps, pharmacies, and police stations are realistic for version one because Places API can help find them.

Private houses are not realistic to count one by one. We should not pretend we can measure house activity accurately.

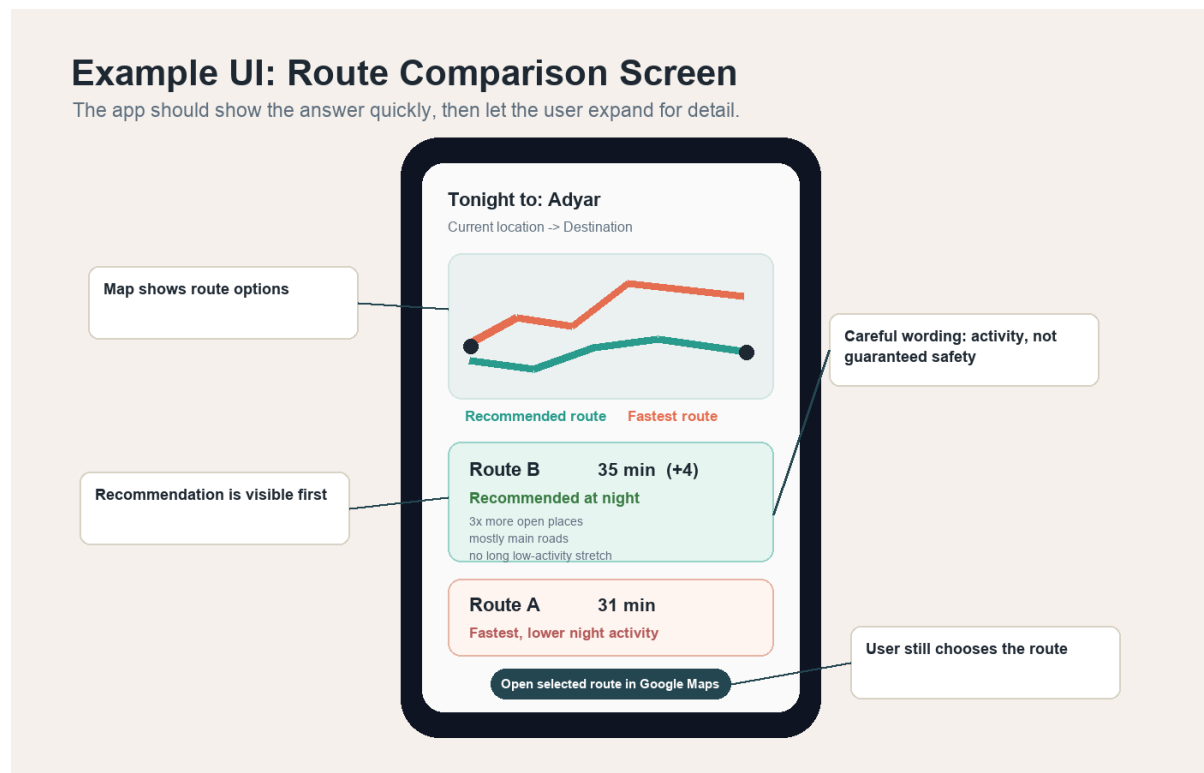
Residential areas can be treated as a soft signal later using OpenStreetMap land-use data.

Street lighting is important, but reliable street-light data may not be available everywhere.

For MVP, we can mention it as a later data layer.

Crime data is sensitive and often incomplete. It should not be used casually in MVP.

Example UI Direction



How The App Actually Works Behind The Screen

The user enters a destination.

The browser asks for current location.

The frontend sends origin and destination to our backend.

The backend asks Routes API for route options.

The backend receives route lines for Route A, Route B, Route C, etc.

The backend breaks each route line into points every 150-200 meters.

For each point, the backend asks Places API what is nearby.

The backend deduplicates repeated places so one shop is not counted five times.

The backend counts open places, staffed places, and useful help points.

The backend detects low-activity stretches.

The backend scores each route.

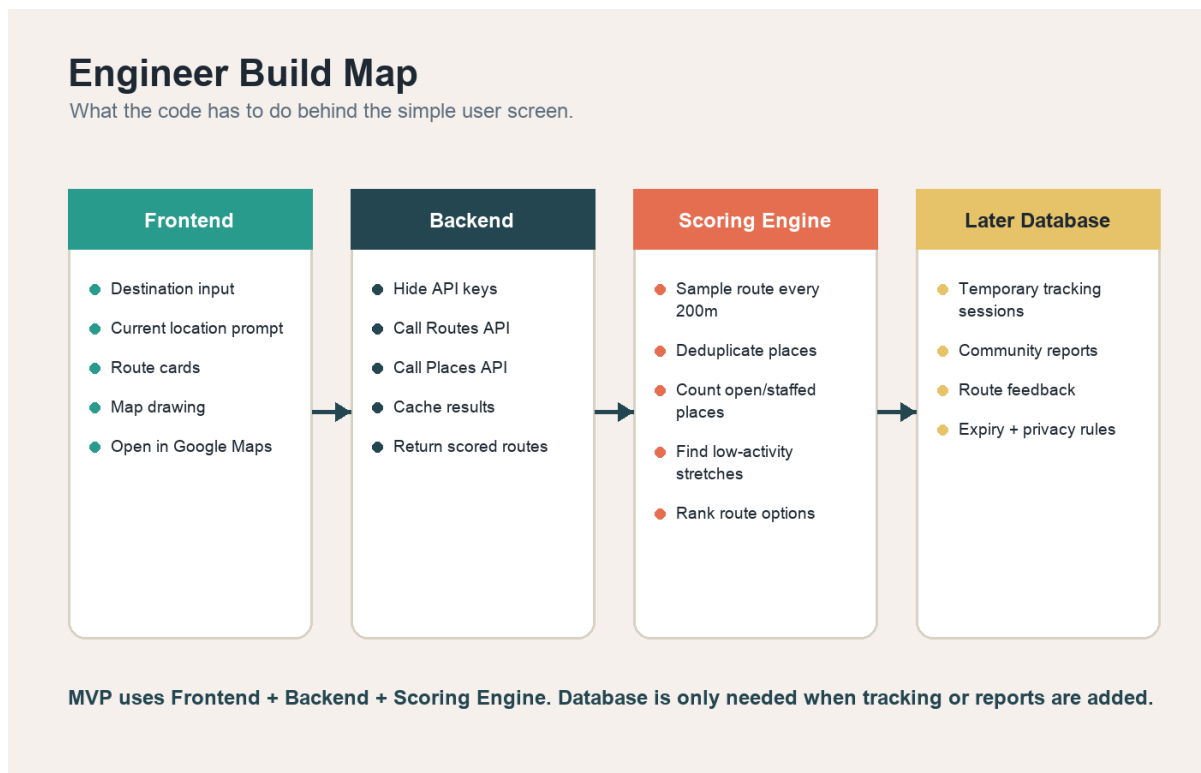
The backend returns the score, explanation, and map data to the frontend.

The frontend shows route cards and map lines.

The user chooses a route.

The app opens that route in Google Maps.

Engineer Build Map



Note for Akash: You can specifically focus on data and activity tracking for areas around the office. Specifically saharakr nagar and areas around manayata tech park so we can actually try it out for the video.

Main App Modules

Module 1: Frontend app

What it does: shows the input screen, map, route cards, explanation, and Google Maps button.

Needs: React or Next.js, Google Maps JavaScript API, browser location permission.

Returns to user: a simple comparison screen.

Module 2: Backend API

What it does: receives origin and destination, calls Google APIs, runs analysis, returns scored routes.

Needs: Node.js backend or Next.js API routes, Google API key, environment variables.

Returns to frontend: route options, scores, explanations, and map geometry.

Module 3: Route fetcher

What it does: asks Google Routes API for 2 to 4 route options.

Input: origin, destination, travel mode.

Output: route ID, duration, distance, polyline, steps.

Module 4: Route sampler

What it does: breaks route lines into regular sample points.

Output: points every 150 to 250 meters.

Why it matters: we cannot judge a route by only checking the start and end. We need to check the journey in between.

Module 5: Places scanner

What it does: checks nearby places around each sample point.

Input: sample point, radius, place categories.

Output: nearby restaurants, cafes, shops, petrol pumps, hospitals, police stations, hotels, pharmacies, transport points, etc.

Module 6: Place deduplicator

What it does: removes repeated places that appear around multiple sample points.

Input: raw places from many API calls.

Output: unique list of places per route.

Why it matters: without this, the score can be fake because the same petrol pump may be counted repeatedly.

Module 7: Activity analyzer

What it does: measures how active a route looks at night.

Input: unique places, open-now status, place type, route points.

Output: open place count, staffed place count, essential help points, activity density.

Module 8: Low-activity stretch detector

What it does: finds the longest part of the route where there are very few open/staffed places nearby.

Input: route sample points and places around each point.

Output: longest low-activity stretch, total low-activity distance.

Why it matters: one long empty stretch is more important than a low average score.

Module 9: Road-type analyzer

What it does: estimates whether the route uses bigger/main roads or smaller/internal roads.

Input: route steps and road names from Google, later OpenStreetMap road tags.

Output: main-road coverage, internal-road penalty, turn complexity.

Module 10: Scoring engine

What it does: combines all signals into one Night Activity Score.

Input: activity count, staffed places, road type, low-activity stretches, time difference.

Output: score out of 100 and rank order.

Module 11: Explanation generator

What it does: turns the score into simple English.

Input: score components and route comparison.

Output: "For 4 extra minutes, this route gives you more open places, mostly main roads, and no long low-activity stretch."

Module 12: Google Maps handoff

What it does: opens the selected route in Google Maps.

Input: origin, destination, selected route details.

Output: Google Maps directions URL.

Scoring Logic

A route gets points for:

More open places near the route.

More staffed/helpful places such as petrol pumps, hospitals, hotels, pharmacies, and police stations.

More main-road coverage.

Fewer long low-activity stretches.

Fewer unnecessary turns through small roads.

More commercial/public activity around the route.

A route loses points for:

Long stretches with very few open places.

Too many small/internal roads.

Too many turns through lanes.

Very few staffed places nearby.

No useful help point for a long distance.

Scoring Formula could be something like:

Open establishments nearby: 25 points.

Main-road coverage: 20 points.

Staffed/helpful places: 15 points.

Low-activity stretch penalty: 15 points.

Route simplicity: 15 points.

Transport/public activity points: 10 points.

Total: 100 points.

This formula should start simple. Akash can adjust weights here.

What The App Should Say To Users

Use:

Recommended at night.

More active route.

Lower nighttime activity.

Long low-activity stretch.

More staffed places nearby.

Mostly main roads.
For 4 extra minutes, you get...

Avoid:
Safe route.
Unsafe route.
Dangerous road.
Crime-free route.
Guaranteed safety.
This route will protect you.

Example Output

Route A - 31 minutes
Fastest route
Night Activity Score: 54/100
Why: fewer open places, more internal roads, and one 1.4 km low-activity stretch.

Route B - 35 minutes
Recommended at night
Night Activity Score: 82/100
Why: more open places, mostly main roads, petrol pumps and hospital nearby, and no long low-activity stretch.

Build Roadmap

Phase 0: Setup
Create GitHub repo.
Create Google Cloud project.
Enable Maps JavaScript API, Routes API, Places API, Places Autocomplete API, and Geocoding API.
Create restricted API keys.
Create .env file for local development.
Deploy a blank app to confirm hosting works.

Phase 1: Basic App Shell
Build one web page.
Add current location button.
Add destination search.
Add "Compare night routes" button.
Show an empty map area and empty route cards.
No scoring yet.

Phase 2: Get Routes

Connect backend to Routes API.

Send origin and destination.

Receive route alternatives.

Show 2 to 4 route options.

Draw routes on the map.

Show time and distance for each route.

Phase 3: Scan Activity

Decode each route polyline.

Sample route every 200 meters.

Call Places API around each sample point.

Search for open/staffed place categories.

Deduplicate repeated places.

Return open place counts and help-point counts.

Phase 4: Detect Dead Stretches

Mark sample points with very low activity.

Group nearby low-activity points together.

Calculate longest low-activity stretch.

Calculate total low-activity distance.

Show this in the route explanation.

Phase 5: Score Routes

Create first version of Night Activity Score.

Compare route scores against fastest route.

Choose recommended night route.

Make sure the score is explainable, not mysterious.

Phase 6: Build Explanation UI

Show the recommended route first.

Show the fastest route as comparison.

Show the extra time difference.

Show 3 to 5 reasons max.

Add expand/collapse for deeper detail.

Phase 7: Google Maps Redirect

Add "Open in Google Maps" button.

Generate directions URL.

Pass origin, destination, and travel mode.

Explain clearly that navigation continues in Google Maps.

Phase 8: Real Route Testing

Test 10 to 20 Chennai routes.

Include known shortcut-vs-main-road examples.

Test after dark where possible.

Check whether Places API returns useful open-now data.

Check if the recommended route makes common sense.

Adjust scoring weights.

Phase 9: Demo Polish

Improve route cards.

Color recommended route clearly.

Add low-activity stretch label on map.

Add simple privacy disclaimer.

Prepare 2 or 3 strong demo routes for the video.

Acceptance Criteria

The app works without login.

The user can enter destination.

The app gets current location after permission.

The app shows at least two route options.

The app shows time and distance for each route.

The app scans nearby places for each route.

The app returns a Night Activity Score.

The app identifies longest low-activity stretch.

The app explains the recommendation simply.

The app lets the user open the selected route in Google Maps.

Known Risks

Google APIs cost money after free credits.

Places API may not always know whether a place is currently open.

Too many Places calls can get expensive.

Some areas may have poor map/place data.

Road type may be hard to infer using only Google data.

Lighting data may not be available.

Crime data can be incomplete, sensitive, and misleading.

Google Maps handoff may not perfectly preserve the exact route line.

How To Reduce Risks

Cache Places results.

Limit route options to 2 to 4.

Sample every 150-200 meters instead of every 20 meters.

Start with one city.

Use careful wording.

Do not claim guaranteed safety.

Test with real routes before making strong product claims.

Testing Routes To Try

Home to office.

Metro station to home.

Mall to home.

Friend's house to home.

Restaurant to home after 10 pm.

Known shortcut route vs main-road route.

Two-wheeler-heavy internal-lane route.

Route from commercial area to residential area.

The first build should stay focused: no login, no community reports, no full navigation system. Just route comparison, night activity reasoning, and Google Maps handoff.

For scripting

<https://chennaicorporation.gov.in/gcc/pdf/The%20Gender%20Compass.pdf>

<https://timesofindia.indiatimes.com/city/chennai/my-safetipin-app-identify-safer-areas-and-routes-for-women-in-chennai/articleshow/108339213.cms>

A good case study on why this tool is needed in the first place. Chennai has this already.

Proposed recommendations for integrating safety features and enhancing user experiences on Google Maps. Coz safety is a complex concept, extending beyond crime rates, and potentially being influenced by factors such as gender, geography, etc.

A male respondent recalled an unsettling experience where Google Maps diverted him to an isolated route when en route to Kalahasti Temple, “Google Maps didn’t show the highway route and instead cut me from going using the highway. I was diverted into a small road. It was dark and I felt like I was going through a forest. I wondered how I would even drive through this. It is unsafe to go with your family at night through such routes.” As shown by the findings of the study, safety in terms of lighting, openness, walk path, people, security, and feeling, presents a deep concern for Google Maps users and has to be taken into consideration in the design and functioning of the application.

<https://reasonstobecheerful.world/get-there-fast-or-safe-a-crowdsourced-map-gives-women-the-option/>

Safetipin

<https://reasonstobecheerful.world/get-there-fast-or-safe-a-crowdsourced-map-gives-women-the-option/>

Social entrepreneur Kalpana Viswanath co-founded the Gurgaon-based company that owns the My Safetipin app, which is backed by several organizations including The Asia Foundation, on the heels of the historic 2012 Nirbhaya rape case in New Delhi.

Viswanath wanted to put “data in the hands of women so that they are empowered to make decisions,” but emphasizes that this alone is not enough. “Technology is only an enabler to bring about change. Technology is not the solution,” she says, noting that the change needed to eradicate violence against women must happen at the policy, law, behavioral and educational levels.